Clase 6: Introducción a R, bash y orquestación

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile

Motivación

- Distintos lenguajes tienen distintas fortalezas (Python, R)
- R es especializado en estadística (interpretado, sin objetos)
- Bash para conectar resultados (NextFlow, Snakemake)

Contents

1. Clases y objetos
2. Polimorfismo

R

Ejecución de R

- Al igual que Python, consola y programa
- Consola

```
$ R          # Abrir programa en bash
> 2 + 2      # Ejecutar en consola de R
[1] 4        # Resultado
> q()        # Salir de consola
```

- Programa

```
$ Rscript test.r   # Ejecutar script con '2+2'
[1] 4              # Resultado
```

Sintaxis: Tipos

- String: `"Hola"`
- Números: `1, 3.5`
- Comentario: `"hola" # string que saluda`
- Imprimir: `print("hola")`
- Asignar variable: `a=2, a<-2, 2->a`
- Variables pueden tener puntos: `.a=2`
- Sumar strings da error!

  `paste("hola","chao")`
- Números son "numeric" por defecto (ver con `class`)
- Booleanos: `TRUE`,`FALSE`

Sintaxis: Tipos II

- Tienen nombres específicos:
  - numeric `4,5.5,-2`
  - integer `1L, -3L, 100L` (si, con la 'L')
  - complex `9+3i`
  - character (strings)
  - logical (booleanos)
- Convertir números:
  `as.numeric()|as.integer()|as.complex()`

Sintaxis II: Operaciones

- Booleanos: `a&b` (and), `a|b` (or), `!a` (not)
- Comparación: `a<b`, `a<=b`, `3==3`, `3!=2`
- Números: `2+2`, `2*3`, `2**3==2^3`, `10%%3`

Sintaxis III: if/else

```
a = 2
b = 3
if (a>b) {
    print("a>b")
}
else if (a==b) {
    print("a==b")
}
else { # a<b
    print("a<b")
}
```

Sintaxis IV: while

```
i <- 0
while (i < 6) {
    i <- i + 1
    if (i == 3) {
      next
    }
print(i)
}
```

Sintaxis V: for

```r
for (x in 1:10) {
  print(x)
}
fruits <- list("apple", "banana", "cherry")

for (x in fruits) {
  print(x)
}
```

Sintaxis VI: functions

```
my_function <- function(x, y=2) {
  return (5 * x * y)
}

print(my_function(3)) # y=2 por defecto
```

Sintaxis VII: Estructuras de datos

- Vector de un solo tipo: `c(1,2,3)`,`1:10`
- Lista: `list(1, 2i, TRUE)`
- Largo: `length(lista)`
- Agregar: `append(lista, "hola")`
- Quitar: `lista[-1]`, `lista[-c(1,3)]`
- Acceso por rango: `lista[2:5]`

*OJO: Listas empiezan en 1.

Sintaxis VIII: Data Frame

```r
# Create a data frame
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
)
Data_Frame

# Todos equivalentes:
Data_Frame[1]
Data_Frame[["Training"]]
Data_Frame$Training
```

Explorando datos

```r
Data.Cars <- mtcars
dim(Data.Cars)
names(Data.Cars)
rownames(Data.Cars)
Data.Cars$cyl  # Comparar con Data.Cars["cyl"]
summary(Data.Cars)
max(iris["Petal.Length"])
which.max(iris$Petal.Length) # con [] no funciona
mean(iris$Petal.Length)
```

Indexaciones y cálculos

Si 'which' solo opera sobre vectores, cómo extraigo por nombre?

```
datos = iris$"Petal.Length"
filas = rownames(iris)
indx_max = which.max(datos)
filas[indx_max]
```

Indexaciones y cálculos

Si 'which' solo opera sobre vectores, cómo extraigo por nombre?

```
datos = iris$"Petal.Length"
filas = rownames(iris)
indx_max = which.max(datos)
filas[indx_max]
```

Estadística

```r
model = lm(Petal.Length ~ Sepal.Length, data = iris)
summary(model)
b = model$coefficients[1]
m = model$coefficients[2]
minx = min(iris$Sepal.Length)
maxx = max(iris$Sepal.Length)
plot(c(minx, maxx), c(b+minx*m, b+maxx*m),
    type="l", col="red") # (l)ine
lines(iris$Sepal.Length, iris$Petal.Length,
      type="p", col="blue") # (p)oint
```

Consola...

Contents

1. Clases y objetos
2. Polimorfismo

Bash

Navegación efectiva

- `cd`, `ls`, etc
- **Usar Tab**

Consola...

Variables

- Tienen validez solo en terminal actual
- Se pueden mantener en sub-terminales con `export`
- Sintaxis sin espacios

```
$ A=2  # Desaparece luego de 'exit' o 'bash'
$ export A=2 # Se mantiene dentro de nuevos procesos 'b
```

if

```
if [ condition ]; then {
    # código por ejecutar
}
elif [ condition2 ]; then {
    # más código
}
else {
    # código final
}
fi
```

Acá, tenemos notación de operaciones booleanas:

- $>, \geq, <, \leq$: `-gt`,`-ge`,`-lt`,`-le`
- `not` (`!`), `and` (`-a`), `or` (`-o`)
- Ver si existe archivo/carpeta/variable: `-e`/`-d`/`-n`[a]

[a] `-n` ve si variable no es string vacío

Ejemplo

```
A=2
if [ ${A} -gt 1 ]; then {
    echo GT
}
else {
    echo LT
}
fi
A=""
if [ -n "${A}" ]; then { # Comillas!
    echo "SI"
}
else {
    echo NO
}
fi
```

Consola

for

```
for n in a b c; do
   echo "$n"
done
```

Wildcards:

- `*.csv` → todos los archivos terminados en csv
- `*.{csv,xls,txt}` → todos los archivos csv, xls, o txt

Consola

Funciones

```
function helloWorld {
  echo "Hello World"
}
# Equivalente:
helloWorld () {
    echo "Hello World"
}
```

Funciones: argumentos

```
function helloWorld {
  echo "$1" # o echo "£{1}"
}
```

- $1, $2, ...: parámetros en orden
- $0: nombre de función
- $#: Número de argumentos
- $@: Todos los argumentos

Consola

source vs bash

- Source: Ejecutar en proceso actual

```
$ source script.sh
```

- Bash: Crear sub-proceso

```
$ sh script.sh # O 'bash script.sh'
```

Consecuencias:

1. se pueden perder variables (`export`)
2. si source sale (`exit`), se cierra consola

Consola

Pipes y redirecciones

```
$ echo "hola"  # Imprime 'hola'
$ # Toma el texto generado por 'echo' y lo pone en 'out.txt'
$ echo "hola" > out.txt
$ # Toma el texto y lo pone al final de 'out.txt'
$ echo "hola" >> out.txt
$ # Toma salida de 'cat out.txt' y lo entrega como input a grep
$ cat 'out.txt' | grep "ho"
$ # Tome la salida del primer comando, 1) la imprime y 2) la guarda en out.txt
$ cat 'in.txt' | tee out.txt  # -a para 'append'
$ # Entrega contenido de 'in.txt' a función 'echo'
$ echo < in.txt
```

Consola

Usar salidas

- Usar el valor de la variable

```
$ echo VARIABLE="${VARIABLE}"
```

- Usar la salida de un código

```
$ A=$(python codigo.py)
$ B=$(head "$A")
```

- Resultado de operaciones (+,-,*,/) [int]

```
$ B=3
$ A=$((2+$B))
```

Orquestación: Ejercicio

Creemos:

1. Un programa en Python que calcula el promedio de números dados
2. Llamemos este programa desde un script en Bash
3. Hagamos que el script en Bash abra un archivo de números y los de a Python
4. Mostrar resultado final

Recap

- En Python/R es fácil crear funciones específicas
- Estas funciones se pueden orquestar desde Bash
- Bash tiene muchas funcionalidades para manipular texto

Clase 6: Introducción a R, bash y orquestación

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile

Mini ejercicios

R:

1. Instalar R en su computador y probar consola
2. Hacer los siguientes ejercicios:

   https://www.w3schools.com/r/r_exercises.asp
3. Se pueden agregar funciones no lineales de parámetros. Pruebe modelos no-lineales:

   ```
   lm(y ~ x + I(x^3) + log(x), data=datos)
   ```

Bash:

- Crear script en bash que lee la variable de ambiente 'A' y dice si es que la variable es mayor a 2.
- Crear script en bash que muestre el nombre y contenido de todos los archivos .txt en la carpeta